Make std::ignore a first-class object

Document #:	P2968R0
Date:	2023-09-09
Project:	Programming Language C++
Audience:	WG21 - Library & Library Evolution

Reply-to:	Peter Sommerlad <peter.cpp@sommerlad.ch>

Contents

§ 1 Abstract

All major open source C++ library implementations provide a suitably implemented `std::ignore` allowing a no-op assignment from any object type. However, according to some C++ experts the standard doesn't bless its use beyond `std::tie`. This paper also resolves issue lwg2933 by proposing to make the use of `std::ignore` as the left-hand operand of an assignment expression official.

§ 2 History

§ 2.1 R0 initial revision

— Adresses LWG 2933

— Takes up some mailing list discussion

§ 3 Introduction

All major C++ open source standard libraries provide similar implementations for the type of `std::ignore`. However, the semantics of `std::ignore`, while useful beyond, are only specified in the context of `std::tie`

§ 3.1 Non-tuple applications of `std::ignore`

Programming guidelines for C++ safety critical systems consider all named functions with a non-void return type similar to having the attribute `[[nodiscard]]`.

As of today, the means to disable diagnostic is a static cast to void spelled with a C-style cast `(void) foo();`. This provides a teachability issue, because, while C-style casts are banned, such a C-style cast need to be taught. None of the guidelines I am aware of, dared to ask for a `static_cast<void>( foo() );` in those situation.

With the semantics provided by the major standard library implementations and the semantics of the example implementation given in the cppreference.com site, it would be much more documenting intent to write

```
std::ignore = foo();
```

instead of the C-style void-cast.

To summarize the proposed change:

1. better self-documenting code telling the intent
2. Improved teachability of C++ to would-be C++ programmers in safety critical environments

§ 3.2 Comparison Table

Code that compiles today will be guaranteed to compile

Before	After
`std::ignore = std::printf("hello ignore\n");` `// compiles but is not sanctioned by the standard`	`std::ignore = std::printf("hello ignore\n");` `// well defined C++`

3.3 LWG issue 2933

This issue asks for a better specification of the type of `std::ignore` by saying that all constructors and assignment operators are constexpr. I don't know if that needs to be said that explicitly, but the assignment operator template that is used by all implementations should be mentioned as being constexpr and applicable to the global constant `std::ignore`.

4 Mailing List discussions

After some initial draft posted to lib-ext@lists.isocpp.org I got some further feedback on motivation and desire to move ignore to a separate header or utility:

Additional motivational usage by Arthur O'Dwyer:

```
  struct DevNullIterator {
    using difference_type = int;
    auto& operator++() { return *this; }
    auto& operator++(int) { return *this; }
    auto operator*() const { return std::ignore; }
  };

  int a[100];
  std::ranges::copy(a, a+100, DevNullIterator());
```

Giuseppe D'Angelo: As an extra, could it be possible to move std::ignore out of `<tuple>` and into `<utility>` or possibly its own header `<ignore>`?

Ville Voutilainen suggested a specification as code, such as:

```
// 22.4.5, tuple creation functions
struct ignore_type { // expostion only
  constexpr decltype(auto) operator=(auto&&) const & { return *this; }
};
inline constexpr ignore_type ignore;
```

or even more brief by me (the lvalue ref qualification is optional, but I put it here):

```
inline constexpr
struct  { // expostion only
  constexpr decltype(auto) operator=(auto&&) const & { return *this; }
} ignore;
```

Thanks to Arthur O'Dwyer for repeating my analysis of the three major open source library implementations. I refrain from adding the implementations here, because implementors will know what they have done and I don't expect them to have to change anything by this paper.

5 Questions to LEWG

Since there was the request to make `std::ignore` available without having to include `<tuple>` the following questions are for LEWG. Note, any yes will require a change/addition to the provided wording and would also put some burden on implementors. An advantage might be slightly reduced compile times for users of `std::ignore` who do not need anything else from header `<tuple>`.

— Should `std::ignore` be made available via its own header `<ignore>` in addition to be available via `<tuple>`? Y/N

— If no: Should `std::ignore` be made available via header `<utility>` in addition to being available via `<tuple>`? Y/N

§ 6 Questions to LWG

— Should the specification of the type of `std::ignore` use code or stick to a more generic text (see below)?

— Do we need to mention “constepxr constructors” as asked for by LWG2933? (see parenthesized text in the Wording section)

§ 7 Impact on existing code

Since `std::ignore` is already implemented in a suitable way in all major C++ standard libraries, there is no impact on existing code.

However, may be LEWG will decide on being more specific and less hand-wavy on the semantics of the type underlying `std::ignore` and even follow the suggestion to move its definition into another header (`<utility>` or `<ignore>`).

If LWG decides on using code for the specification, libraries might want to adjust their implementation accordingly (which I believe is not required).

§ 8 Wording

In [tuple.syn] change the type of ignore from “unspecified” to an exposition-only type

```
// 22.4.5, tuple creation functions
+struct ignore_type; // expostion only
-inline constexpr unspecified ignore;
+inline constexpr ignore_type ignore;
```

Add at the end of [tuple.syn] the following paragraph:

The exposition-only class *`ignore_type`* provides (constexpr constructors (implicitly) and) a constexpr const assignment operator template that allows assignment to `ignore` from any non-void type without having an effect.

§ 9 References

— LWG 2933

— The markdown of this document is available at my github